Universidad Rafael Landívar
Facultad de ingeniería
Manejo e implementación de archivos
Sección: 02
Ing. Luis Enrique Rojas

LABORATORIO 1 – SEMANA #14
INTEGRACIÓN SERVICIOS EN LA NUBE

Shayla Mayré De León Monterroso
Carné: 1342925

Guatemala de la Asunción, 07 de octubre de 2026

Laboratorio No.1 – Semana 14 - Integración servicios en la nube

Nombre: Shayla Mayré De León Monterroso **Carné**: 1342925

Objetivo: Desarrollar y publicar una aplicación web que permita interactuar con archivos almacenados en Azure Blob Storage, utilizando control de versiones con Git y una canalización CI/CD de Azure DevOps para automatizar el despliegue de la aplicación en Azure App Service

Resultado esperado

- Aplicación web funcional ejecutándose en Azure App Service.
- Conexión entre App Service y Azure Blob Storage.
- Listado y consulta de archivos almacenados en un contenedor Blob.
- Operación de carga de archivos al Blob Storage.
- Repositorio Git con historial de cambios.
- Pipeline de Azure DevOps que compile y despliegue automáticamente la aplicación.
- Evidencias que demuestren el funcionamiento de la solución.

Repositorio recomendado
Utilice como punto de partida el ejemplo oficial de Microsoft/Azure Samples revisado para esta actividad:
GitHub: https://github.com/Azure-Samples/serviceconnector-webapp-storageblob-dotnet
El tutorial oficial utiliza una aplicación web .NET conectada a Azure Blob Storage mediante una identidad administrada y Service Connector. La documentación indica además que la aplicación puede desplegarse en Azure App Service.

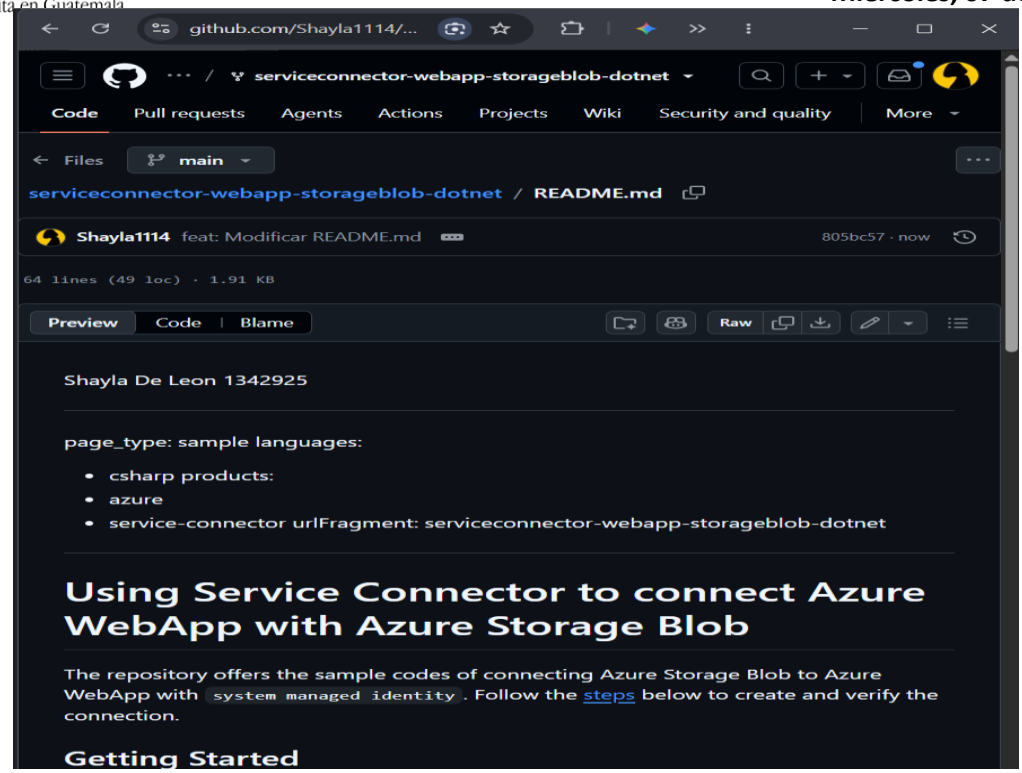
Tecnologías

Tecnología	Uso
C# / .NET	Desarrollo de la aplicación web.
Azure Blob Storage	Almacenamiento de archivos.
Azure App Service	Hospedaje de la aplicación.
Git	Control de versiones.
Azure DevOps Pipelines	Integración y despliegue continuo.
Managed Identity / Service Connector	Acceso seguro desde App Service al almacenamiento.

Realización de la actividad

1. Preparar el repositorio

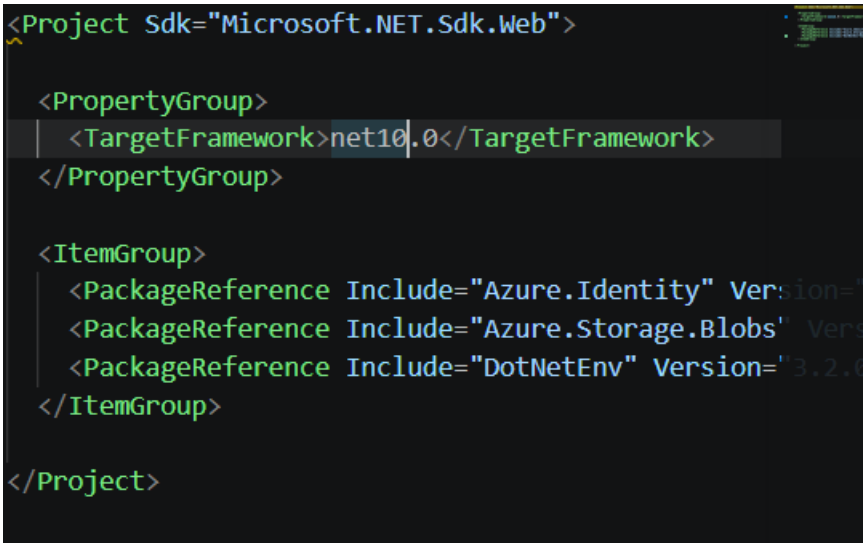
En el repositorio brindado arriba, realice un fork del repositorio base o copie el proyecto a un repositorio propio. El repositorio debe pertenecer al equipo de trabajo. Realice un primer commit identificable



---

2. Revisar la aplicación:
Ejecute la aplicación localmente y revise cómo obtiene acceso al Blob Storage. Identifique el punto del código donde se realiza la lectura/escritura de blobs y documente brevemente su funcionamiento. Visual Studio -> carpeta -> MIA_Semana14 -> git clone [repo url] -> dotnet commands

NOTA: cambia ren .csproj file -> <TargetFramework>net9.0</TargetFramework>

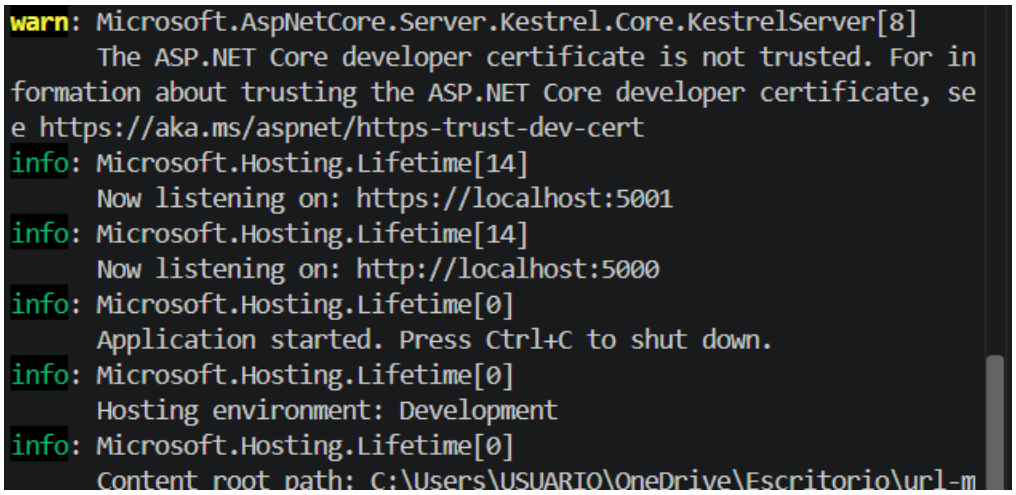


---

3. Crear DotNet Env
a) En terminal, instalar: **dotnet add package DotNetEnv**
b) Crear .env file en raíz de proyecto, valores ->

AZURE_STORAGEBLOB_RESOURCEENDPOINT=https://miarchivos2026.blob.core.windows.net/
AZURE_STORAGE_CONTAINER_NAME=default

c) Crear .env.production con los mismos valores en el mismo lugar
d) Anadir cambio brindado en program.cs
e) dotnet clean -> dotnet restore -> dotnet build -> dotnet run



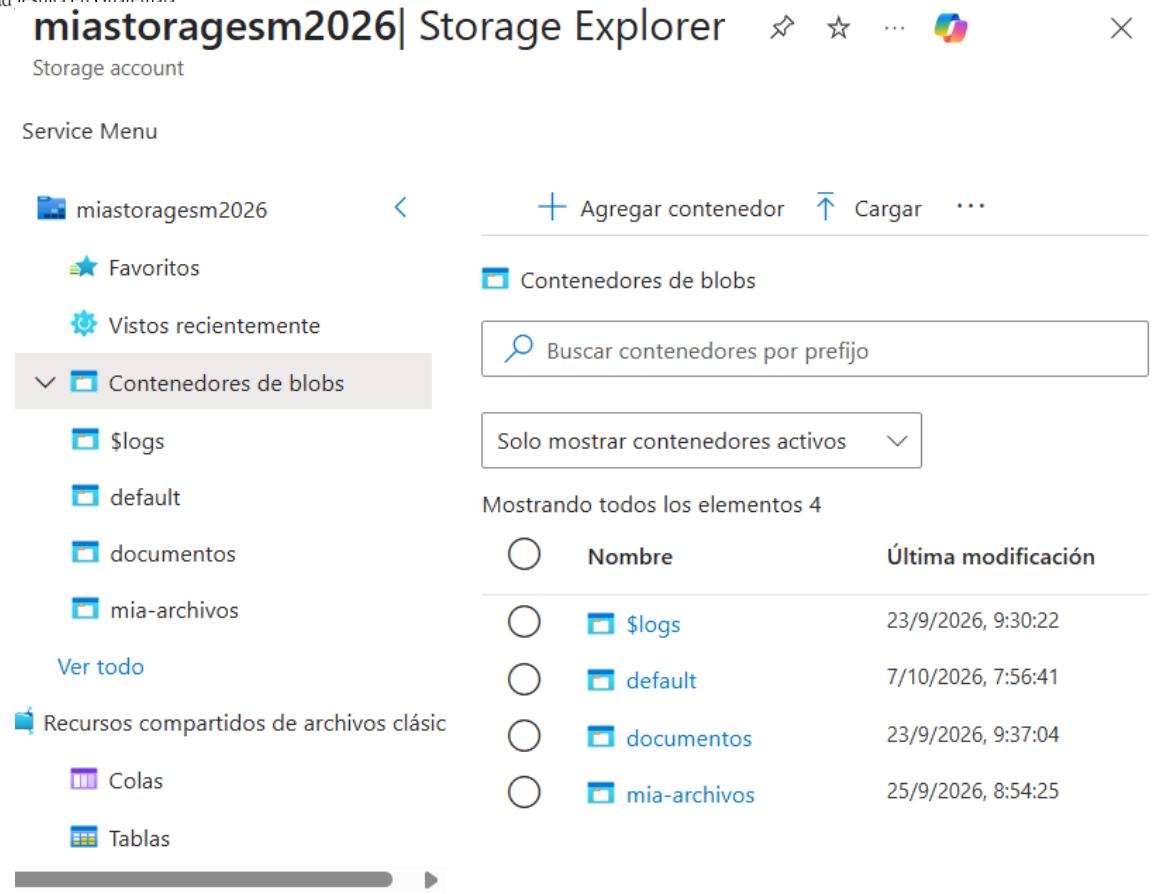
4. Crear recursos en Azure

En azure portal site, cree o utilice un Resource Group, una Storage Account con Blob Storage y un contenedor. El contenedor deberá tener al menos tres archivos de prueba, Puede utilizar el creado en semana 13.

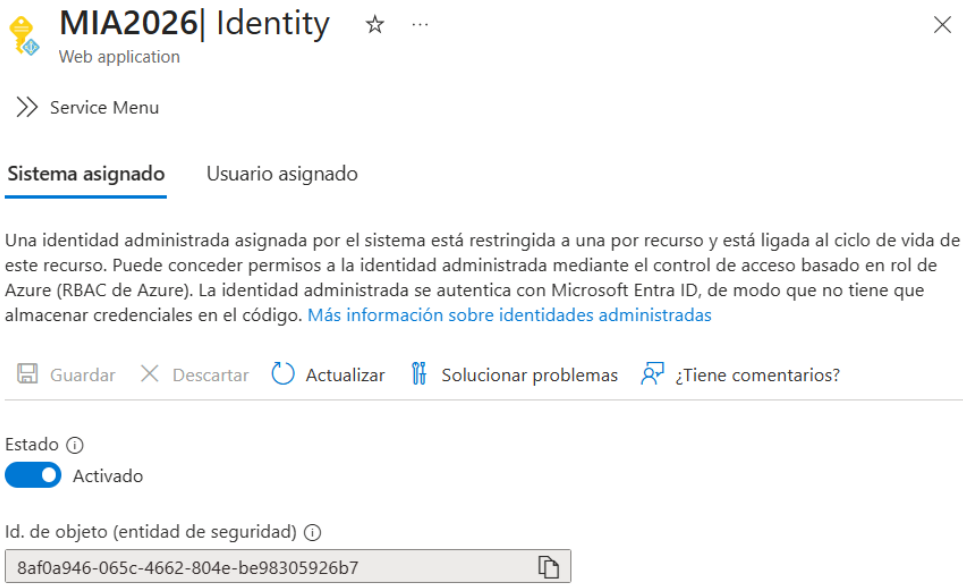
5. Conectar App Service con Blob Storage

Configure la conexión entre App Service y Blob Storage mediante Service Connector utilizando identidad administrada. Verifique que la identidad tenga los permisos necesarios sobre los blobs.

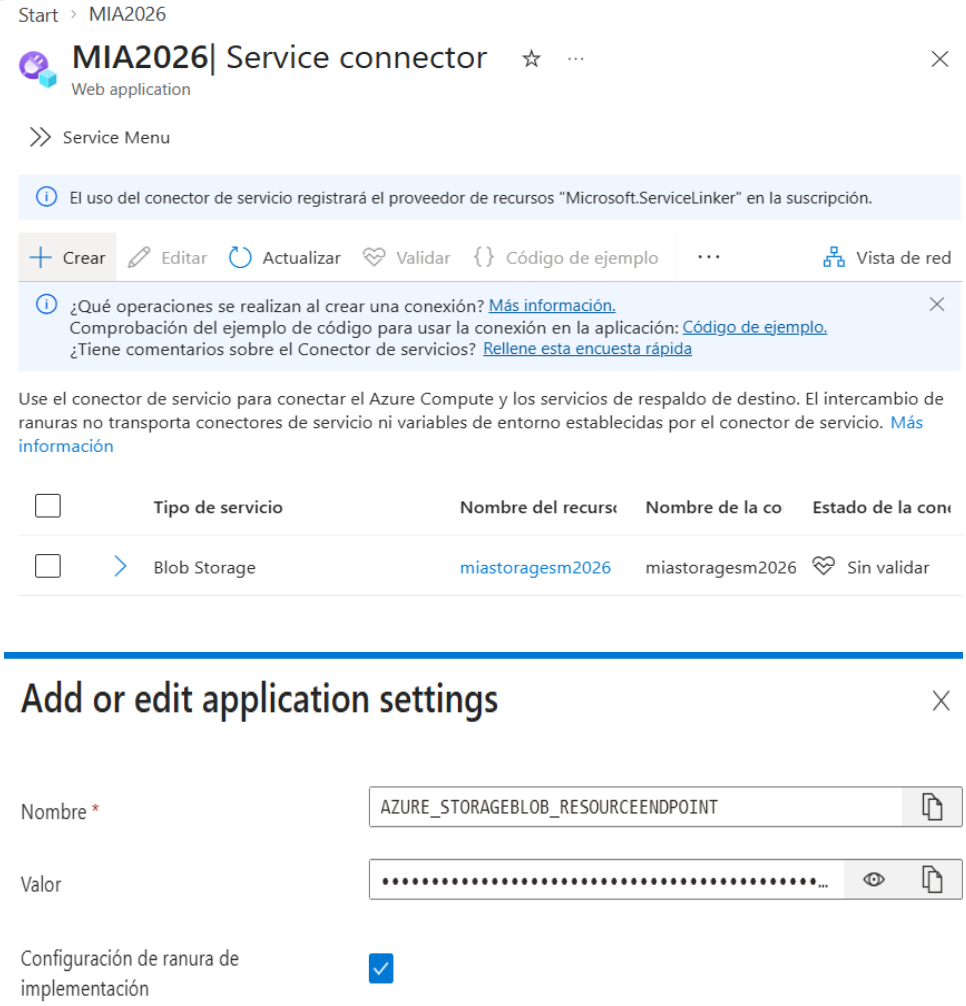
a) Azure Portal → **Storage accounts → tu Storage Account → Data storage → Containers**
b) Crear contenedor “default



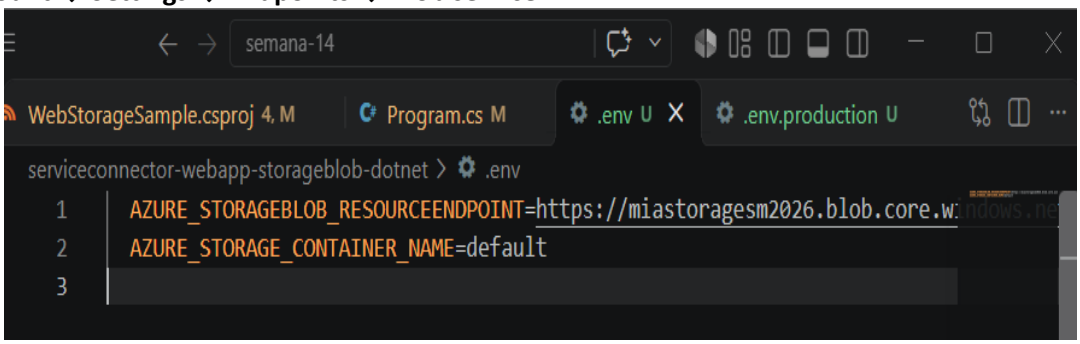
c) Activa identidad administrada en App Service: **Azure Portal → App Service → Settings → Identity → System assigned: on → Save (web app creada semana 13)**



d) Configurar la conexión entre App Service y Blob Storage: **App Service → Settings → Service Connector → Create → Seleccionar "Storage Blob" → Seleccionar Storage Account [tu_blob_name] → Authentication "System-assigned managed identity" → Review + Create**.



e) Reemplazar storage blob endpoint y container nombre en .env file
Storage Account → Settings → Endpoints → Blob service



---

6. Probar el acceso al almacenamiento:

Ejecute la aplicación local y compruebe que puede leer los blobs. Realice al menos una operación de carga de archivo y verifique desde Azure Portal que el archivo llegó al contenedor. Visual Studio -> dotnet run

a) Verifique que Azure CLI esté instalado ejecutando en la terminal de VS Code: az –version

```
lob-dotnet> az --version
azure-cli                         2.91.0

core                              2.91.0
telemetry                          1.1.0

Dependencies:
msal                              1.39.0
azure-mgmt-resource               24.0.0

Python location 'C:\Program Files\Microsoft SDKs\Azure\CLI2\python.exe'
Config directory 'C:\Users\USUARIO\.azure'
Extensions directory 'C:\Users\USUARIO\.azure\cliextensions'

Python (Windows) 3.14.7 (tags/v3.14.7:823f032, Aug  5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64
)]

Legal docs and information: aka.ms/AzureCliLegal


Your CLI is up-to-date.
```

b) Inicie sesión en Azure: az login

c) Verifique la cuenta de Azure activa: az account show

```
PS C:\Users\USUARIO\OneDrive\Escritorio\url-mia-2026\semana-14\serviceconnector-webapp-storageb
lob-dotnet> az account show
{
  "environmentName": "AzureCloud",
  "homeTenantId": "0f78549d-3eec-43af-b56a-6f7b042d6c9a",
  "id": "c0200ed1-deba-4ead-8039-71ee663b21e2",
  "isDefault": true,
  "managedByTenants": [],
  "name": "Azure for Students",
  "state": "Enabled",
  "tenantDefaultDomain": "correo.url.edu.gt",
  "tenantDisplayName": "Universidad Rafael Landivar",
  "tenantId": "0f78549d-3eec-43af-b56a-6f7b042d6c9a",
  "user": {
    "name": "smdeleonmon@correo.url.edu.gt",
    "type": "user"
  }
}
```

d) Verifique que pueda obtener un token para Azure Storage: az account get-access-token --resource https://storage.azure.com/

```
PS C:\Users\USUARIO\OneDrive\Escritorio\url-mia-2026\semana-14\serviceconnector-webapp-storageb
lob-dotnet> az account get-access-token --resource https://storage.azure.com/
{
  "accessToken": "eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiIsIng1dCI6ImRndlNEdks4QTVLeUt5cHB3MWRBd1RYR
DNDQSIsImtpZCI6ImRndlNEdks4QTVLeUt5cHB3MWRBd1RYRDNDQSJ9.eyJhdWQiOiJodHRwczovL3N0b3JhZ2UuYXp1cmU
uY29tLyIsImlzcyI6Imh0dHBzOi8vc3RzLndpbmRvd3MubmV0LzBmNzg1NDlkLTNlZWMtNDNhZi1iNTZhLTZmN2IwNDJkNm
M5YS8iLCJpYXQiOjE3OTE0MjExMzIsIm5iZiI6MTc5MTQyMTEzMiwiZXhwIjoxNzkxNDI1ODk0LCJhY3IiOiIxIiwiYWlvI
```

e) Si se muestra un accessToken la autenticación esta correcta, ejecutar -> dotnet run

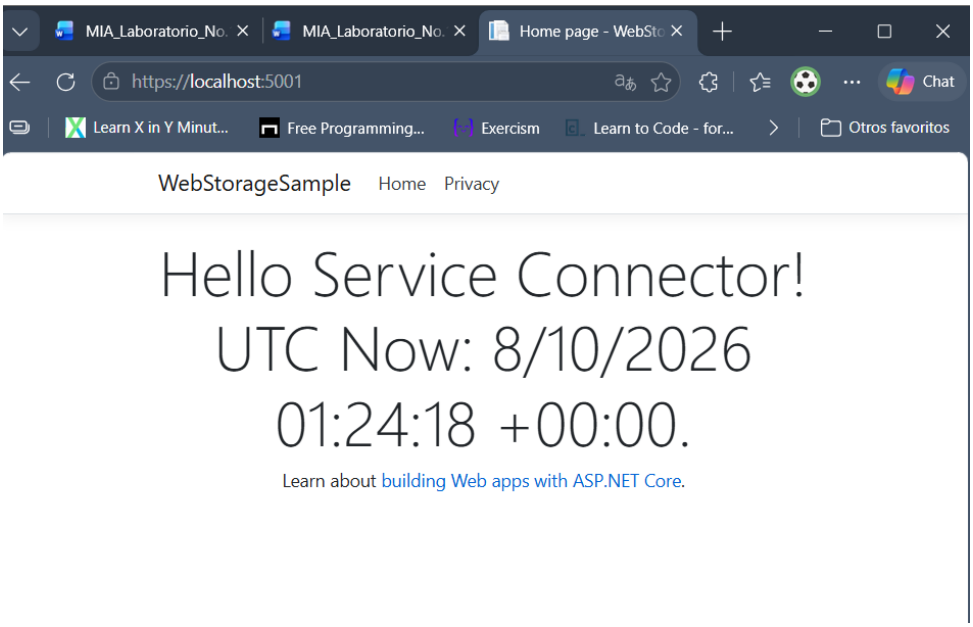
f) Si hay error en acceso, ejecutar en terminarl vs code: az role assignment create --assignee "TU_CORREO_DE_AZURE" --role "Storage Blob Data Contributor" --scope "/subscriptions/TU_SUBSCRIPTION_ID/resourceGroups/TU_RESOURCE_GROUP/providers/Microsoft.Storage/storageAccounts/TU_STORAGE"

```
PS C:\Users\USUARIO\OneDrive\Escritorio\url-mia-2026\semana-14\serviceconnector-webapp-storageb
lob-dotnet> az account show --query "{subscriptionId:id, user:user.name}" -o table
SubscriptionId                        User
------------------------------------  --------------------------
c0200ed1-deba-4ead-8039-71ee663b21e2  smdeleonmon@correo.url.edu.gt
PS C:\Users\USUARIO\OneDrive\Escritorio\url-mia-2026\semana-14\serviceconnector-webapp-storageb
lob-dotnet> az storage account show --name miastoragesm2026 --query "{id:id, resourceGroup:reso
urceGroup}" -o table
ResourceGroup
--------------
rg-mia-lab
PS C:\Users\USUARIO\OneDrive\Escritorio\url-mia-2026\semana-14\serviceconnector-webapp-storageb
lob-dotnet>
```

```
PS C:\Users\USUARIO\OneDrive\Escritorio\url-mia-2026\semana-14\serviceconnector-webapp-storageblob-dotnet> az role assignment create --assignee "smdeleonmon@correo.url.edu.gt" --role "Storage Blob Data Contributor" --scope "/subscriptions/c0200ed1-deba-4ead-8039-71ee663b21e2/resourceGroups/rg-mia-lab/providers/Microsoft.Storage/storageAccounts/miastoragesm2026"
{
  "condition": null,
  "conditionVersion": null,
  "createdBy": null,
  "createdOn": "2026-10-08T01:21:25.311296+00:00",
  "delegatedManagedIdentityResourceId": null,
  "description": null,
  "id": "/subscriptions/c0200ed1-deba-4ead-8039-71ee663b21e2/resourceGroups/rg-mia-lab/providers/Microsoft.Storage/storageAccounts/miastoragesm2026/providers/Microsoft.Authorization/roleAssignments/a2cf6bd9-2815-48c4-844f-64e864276ae6",
  "name": "a2cf6bd9-2815-48c4-844f-64e864276ae6",
  "principalId": "5cf03fc9-8aaf-4b05-bdb6-4cff19941dc1",
  "principalType": "User",
  "resourceGroup": "rg-mia-lab",
  "roleDefinitionId": "/subscriptions/c0200ed1-deba-4ead-8039-71ee663b21e2/providers/Microsoft.Authorization/roleDefinitions/ba92f5b4-2d11-453d-a403-e96b0029c9fe",
  "scope": "/subscriptions/c0200ed1-deba-4ead-8039-71ee663b21e2/resourceGroups/rg-mia-lab/providers/Microsoft.Storage/storageAccounts/miastoragesm2026",
  "systemData": null,
  "type": "Microsoft.Authorization/roleAssignments",
  "updatedBy": "5cf03fc9-8aaf-4b05-bdb6-4cff19941dc1",
  "updatedOn": "2026-10-08T01:21:25.462301+00:00"
}
```

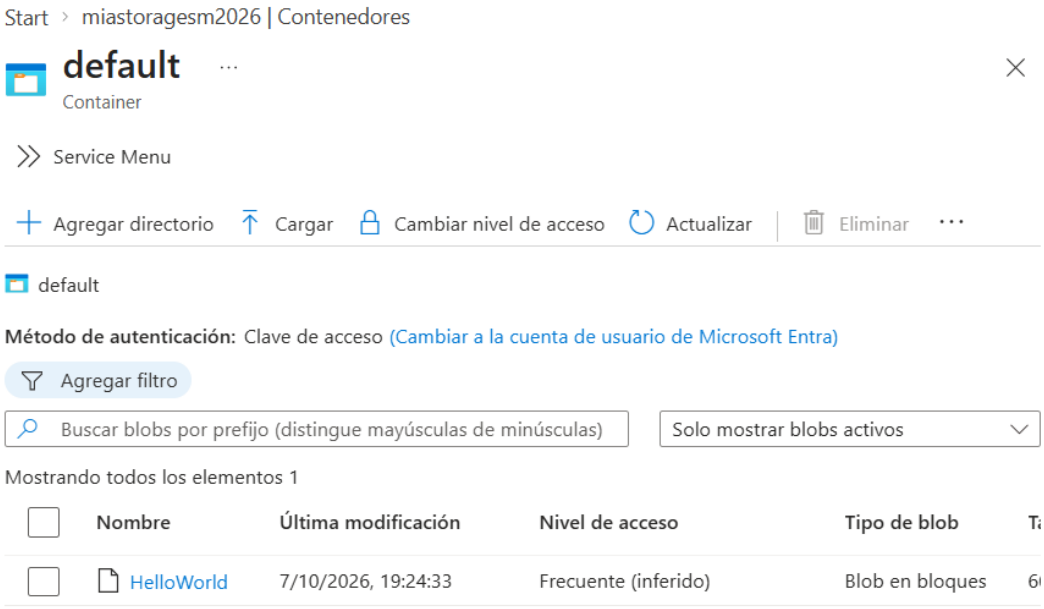
Obtener info: az account show --query "{subscriptionId:id, user:user.name}" -o table
az storage account show --name “tu_storage” --query "{id:id, resourceGroup:resourceGroup}" -o table

g) Cerrar process ctr +c y logear de nuevo: az logout y az login, Por ultimo dotnet run



R// La aplicación utiliza la clase StorageHelper para interactuar con Azure Blob Storage. El método UploadBlob crea o accede al contenedor y carga contenido en un blob, mientras que GetBlob obtiene un archivo existente y lee su contenido. Ambos métodos se autentican mediante DefaultAzureCredential, evitando utilizar claves de acceso directamente en el código.

R// En el archivo Index.cshtml.cs, el método OnGet() genera un mensaje con la fecha y hora actual. Luego utiliza StorageHelper.UploadBlob() para almacenar el contenido en Azure Blob Storage y posteriormente StorageHelper.GetBlob() para leerlo nuevamente. El valor recuperado se asigna a DisplayWords, lo que permite mostrar en la página web la información obtenida desde el almacenamiento.



Nota: no olvidar subir cambios a repo github fork

---

7. Crear Pipeline Yml

En la raíz de tu fork de GitHub crear archivo “azure-pipelines.yml”
Colocar en el archivo lo siguiente:

```
trigger:
- main

pool:
  vmImage: 'windows-latest'

variables:
  buildConfiguration: 'Release'
  dotnetVersion: '9.x'

  # Completar posteriormente
  azureSubscription: 'NOMBRE-DE-TU-SERVICE-CONNECTION'
  appName: 'NOMBRE-DE-TU-APP-SERVICE'

steps:

# 1. Instalar .NET 9
- task: UseDotNet@2
  displayName: 'Instalar .NET 9 SDK'
  inputs:
    packageType: 'sdk'
    version: '$(dotnetVersion)'

# 2. Restaurar dependencias
- task: DotNetCoreCLI@2
  displayName: 'Restaurar dependencias'
  inputs:
    command: 'restore'
    projects: '**/*.csproj'
```

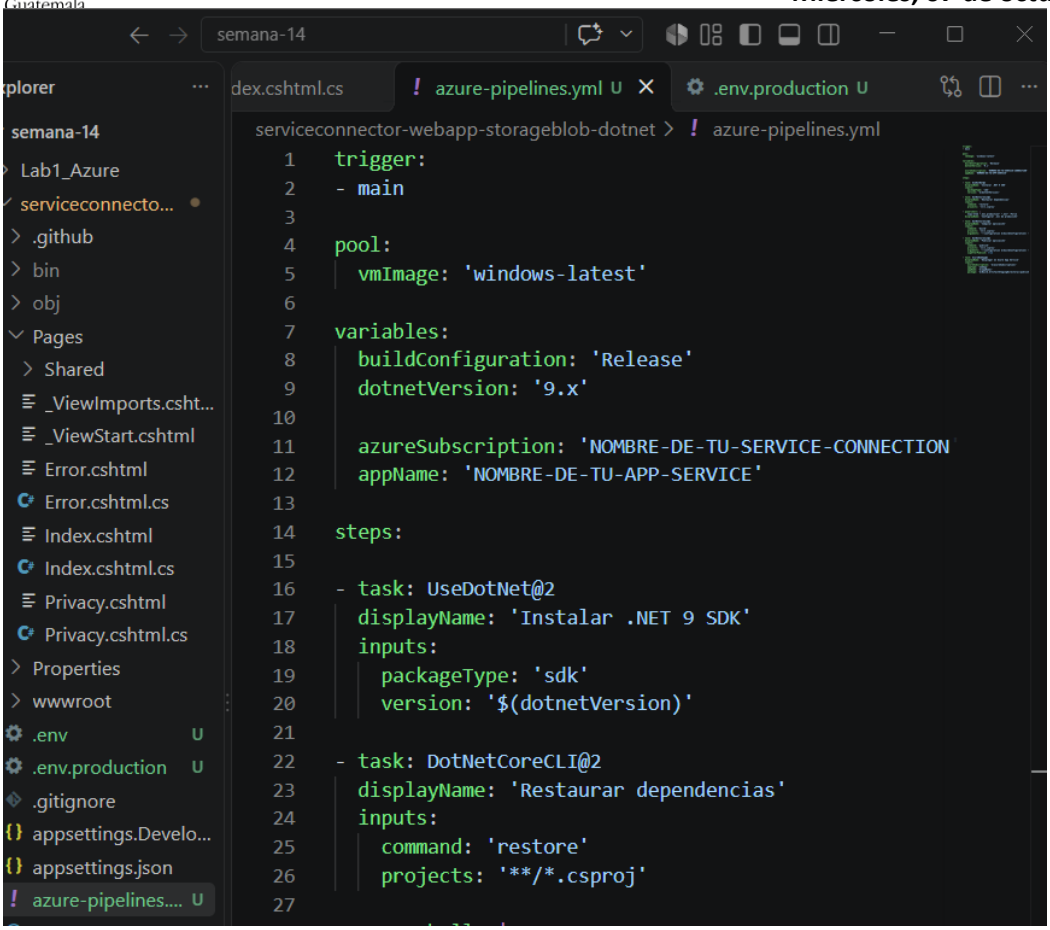
```
# 3. Configurar ambiente de producción
- powershell: |
    Copy-Item ".env.production" ".env" -Force
  displayName: 'Configurar .env de producción'

# 4. Compilar
- task: DotNetCoreCLI@2
  displayName: 'Compilar aplicación'
  inputs:
    command: 'build'
    projects: '**/*.csproj'
    arguments: '--configuration $(buildConfiguration) --no-restore'




# 5. Publicar
- task: DotNetCoreCLI@2
  displayName: 'Publicar aplicación'
  inputs:
    command: 'publish'
    projects: '**/*.csproj'
    arguments: '--configuration $(buildConfiguration) --output $(Build.ArtifactStagingDirectory)/publish --no-
build'
    zipAfterPublish: true

# 6. Desplegar en Azure App Service
- task: AzureWebApp@1
  displayName: 'Desplegar en Azure App Service'
  inputs:
    azureSubscription: '$(azureSubscription)'
    appType: 'webApp'
    appName: '$(appName)'
    package: '$(Build.ArtifactStagingDirectory)/publish/**/*.zip'
```



---

8. Crear pipeline CI/CD

Cree un pipeline YAML que restaure dependencias, compile, publique y despliegue la aplicación en Azure App Service.

a) Conectarse a Azure Devops son misma cuenta que azure portal -> https://aex.dev.azure.com/me

b) Crear pipeline: **En Azure devOps -> Menu izquierda -> pipelines -> Create Pipeline -> Seleccionar Github -> Autoriza Azure DevOps para acceder a GitHub -> seleccionar repositorio que hizo fork -> seleccionar Existing Azure Pipelines YAML file -> Branch: main Path: /azure-pipelines.yml**

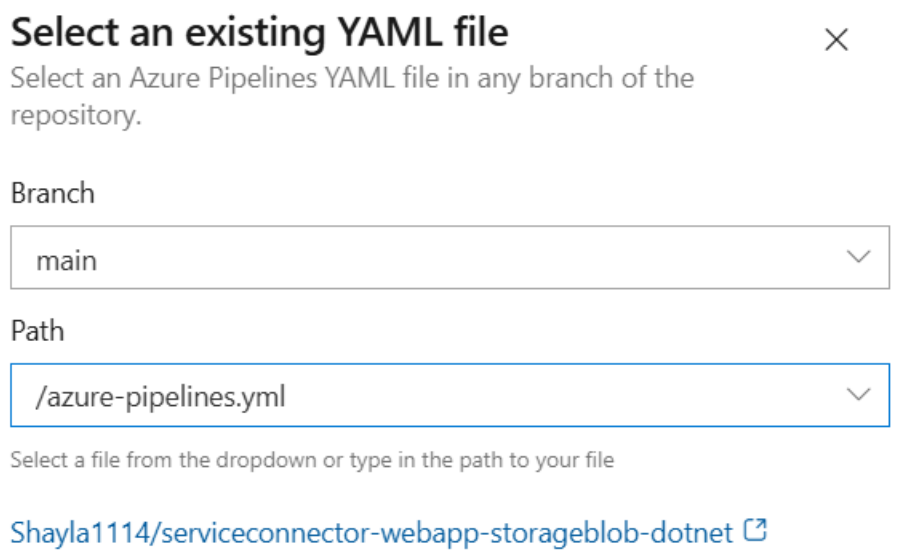
c) Probar seleccionando “run”

d) Conectar Azure DevOps con Azure: **Project Settings → Service connections → New service connection -> Azure Resource Manager -> luego, Workload identity federation -> Selecciona tu suscripción de Azure -> y resource group donde estan tus recursos -> Anadir nombre “MIA_Connection” -> Seleccionar “grant access permission to all pipelines” -> Save**

```
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   .env.production
        modified:   Program.cs
        modified:   WebStorageSample.csproj
        new file:   azure-pipelines.yml

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .env

PS C:\Users\USUARIO\OneDrive\Escritorio\url-mia-2026\semana-14\servicec
onnector-webapp-storageblob-dotnet> git commit -m "feat: configurar Azu
re Blob Storage y pipeline"
[main c0281cc] feat: configurar Azure Blob Storage y pipeline
 4 files changed, 69 insertions(+), 9 deletions(-)
 create mode 100644 .env.production
 create mode 100644 azure-pipelines.yml
PS C:\Users\USUARIO\OneDrive\Escritorio\url-mia-2026\semana-14\servicec
onnector-webapp-storageblob-dotnet> git push
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 16 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 1.28 KiB | 656.00 KiB/s, done.
Total 6 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To https://github.com/Shayla1114/serviceconnector-webapp-storageblob-do
tnet.git
   805bc57..c0281cc  main -> main
```



---

8. Conectar Pipeline con App

a) Obtener nombre app: En Azure Portal App Services → tu aplicación → Copia nombre app

b) en github en tu “azure-pipelines.yml” file busca: azureSubscription y appName, cambiar por valores reales.

azureSubscription: TU_Service_Connection_name
appName: Tu _app_name

c) Al hacer commit, verificar el pipeline se ejecuta automáticamente

New pipeline

Review your pipeline YAML

Variables | Run

Shayla1114/serviceconnector-webapp-storageblob-dot... / azur... | Show assistar

```yaml
trigger:
- main

pool:
  vmImage: 'windows-latest'

variables:
  buildConfiguration: 'Release'
  dotnetVersion: '9.x'

  # Completar posteriormente
  azureSubscription: 'NOMBRE-DE-TU-SERVICE-CONNECTION'
  appName: 'NOMBRE-DE-TU-APP-SERVICE'

steps:

# 1. Instalar .NET 9
Settings
- task: UseDotNet@2
  displayName: 'Instalar .NET 9 SDK'
  inputs:
    packageType: 'sdk'
    version: '$(dotnetVersion)'

# 2. Restaurar dependencias
Settings
- task: DotNetCoreCLI@2
```

New Azure service connection

Azure Resource Manager

Workload identity federation — Recommended

Scope level

- (•) Subscription
- ( ) Management Group
- ( ) Machine Learning Workspace

Subscription

Azure for Students (c0200ed1-deba-4ead-8039-71ee663b2...

If you can't find subscription you need, please create manually.

Resource group

rg-mia-lab

Specify to limit access to the chosen resource group only.

Service Connection Name

MIA_Connection

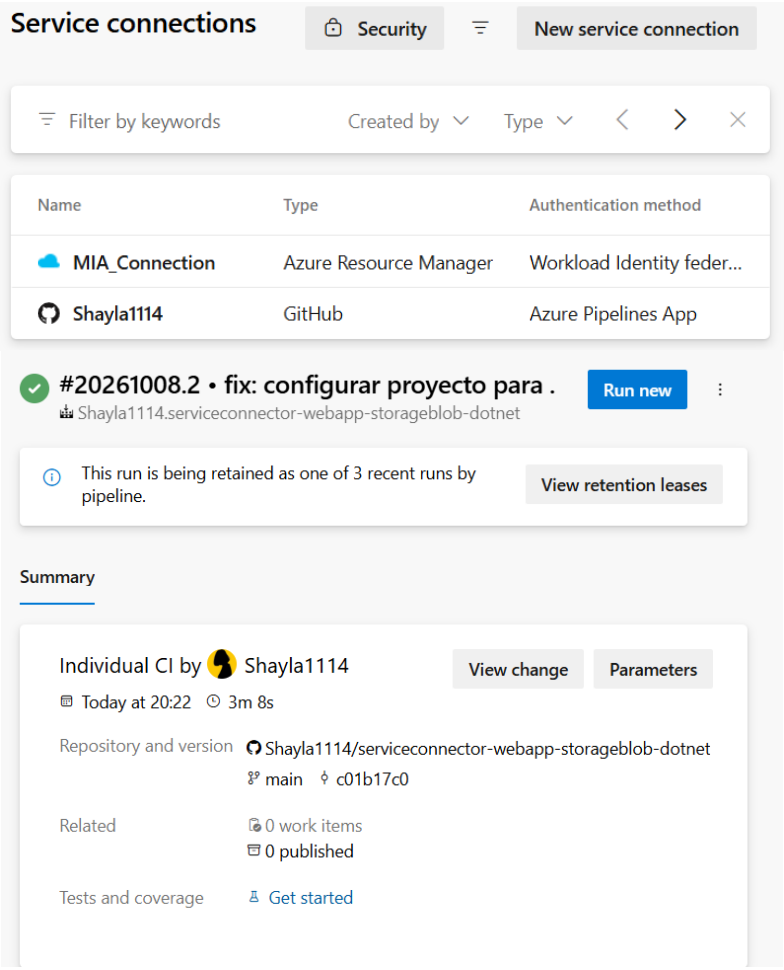
Service Management Reference (optional)

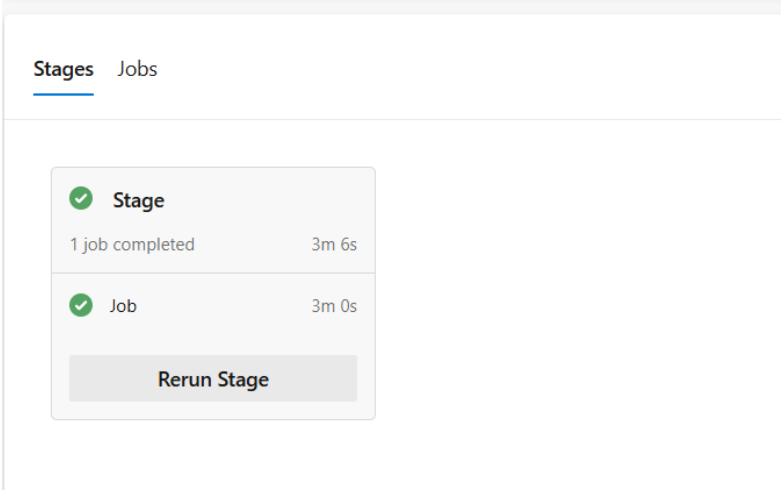
Provide a Service Management Reference if your organization requires one to be configured on Entra ID App registrations.

Description (optional)

Security

[x] Grant access permission to all pipelines





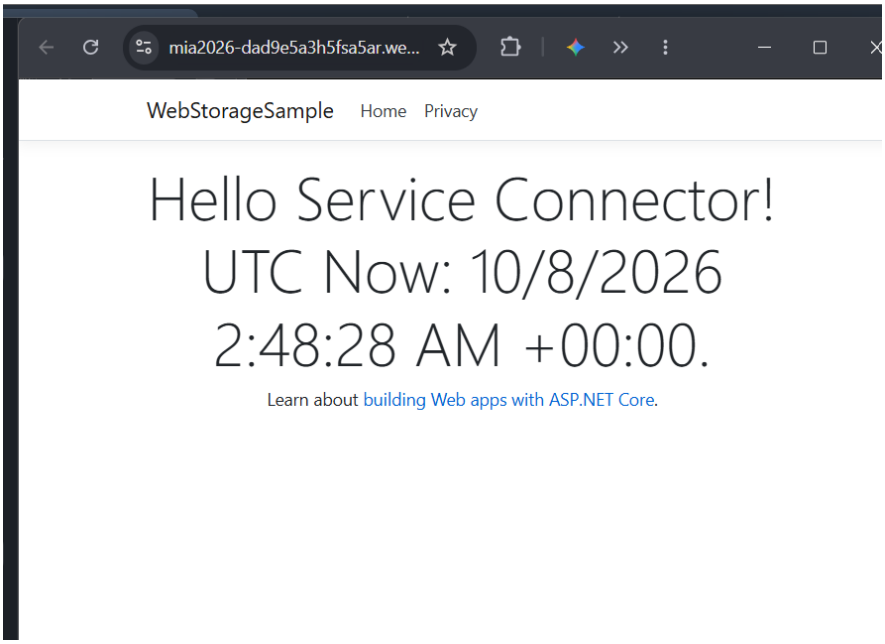
---

9. Validar la solución

Compruebe desde la URL pública que la aplicación funciona y que el acceso al Blob Storage continúa operativo después del despliegue.

a) acceso de la aplicación al Blob: Azure Portal → App Service → Settings → Environment variables → App settings -> colocar AZURE_STORAGEBLOB_RESOURCEENDPOINT
https://tu_storage.blob.core.windows.net/
(si existe obviar)

b) Verificar app: Azure Portal → App Service → OverView → Default Domain → click en URL publica
c) En la app publicada realizar cambios en archivos
d) Verificar que los cambios se representen en storage account: Azure Portal → Storage Account → Storage Browser → Blob containers → Seleccionar su container → Verificar cambios



---

10. Preguntas de analisis

1. ¿Para qué sirve el archivo .env en la aplicación?
   R// El archivo. env se utiliza para almacenar variables de configuración que necesita la aplicación, como el endpoint de Azure Blob Storage y el nombre del contenedor. Esto permite mantener estos valores separados del código fuente y facilita su modificación según el entorno de ejecución.
2. ¿Por qué utilizar Blob Storage y no guardar los archivos dentro del proyecto?
   R// Azure Blob Storage permite almacenar los archivos de forma independiente a la aplicación. De esta manera, los archivos pueden mantenerse disponibles, aunque la aplicación se actualice o vuelva a desplegarse, además de facilitar su administración desde los servicios de Azure.
3. ¿Cuál es la diferencia entre el endpoint de Blob Storage y la URL de App Service?
   R// El endpoint de Blob Storage corresponde a la dirección del servicio donde se almacenan y consultan los archivos. Por otro lado, la URL de App Service es la dirección pública utilizada por el usuario para acceder a la aplicación web desplegada en Azure.
4. ¿Por qué utilizar una identidad administrada en lugar de una Storage Account Key?
   R// La identidad administrada permite que la aplicación se autentique en Azure sin necesidad de guardar claves de acceso directamente en el código o en archivos de configuración. Esto mejora la seguridad y permite administrar los permisos mediante roles asignados en Azure.
5. ¿Qué función cumplen GitHub, Azure DevOps y la Service Connection?
   R// GitHub se utiliza para almacenar el código fuente y mantener el historial de cambios mediante Git. Azure DevOps se encarga de ejecutar la canalización CI/CD para restaurar dependencias, compilar, publicar y desplegar la aplicación. La Service Connection permite que Azure DevOps tenga autorización para conectarse con los recursos de Azure y realizar el despliegue.
6. ¿Qué ocurre después de hacer push a main?
   R// Después de realizar un push a la rama main, el cambio se envía al repositorio de GitHub. Como el archivo azure-pipelines.yml está configurado para ejecutarse con cambios en main, Azure DevOps inicia automáticamente el pipeline para restaurar dependencias, compilar, publicar y desplegar la aplicación.
7. Si el pipeline termina correctamente, pero la aplicación recibe un 403 al subir un archivo, ¿qué revisarías?

R// Revisaría principalmente los permisos de acceso al Blob Storage. Verificaría que la identidad utilizada por la aplicación tenga asignado un rol adecuado, como Storage Blob Data Contributor, y que la conexión entre App Service y Blob Storage esté correctamente configurada mediante Service Connector.

Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-14/lab1/ y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.

Enviar link de github a plataforma de la universidad.